

Module 2

Design & Verification Methodology (Part 2)

Learning objectives

- Basic TB vs UVM: From “wiggle pins and check” to “transaction → sequence → driver/monitor → scorebo...
- UVM building blocks: Transaction (sequence item), sequence, driver, monitor, scoreboard, agent, env...
- Toolchain: UVM_HOME, include paths, Verilator flags (``-sv``, ``--timing``, ``--trace``), and ``make SIM=v...`

Prerequisites

- Module 1 completed (spec → RTL flow, spec_to_rtl example run).
- Verilator (5.036+)
- GNU Make and C++ compiler
- UVM: UVM_HOME set (must contain `src/uvm\_pkg.sv`) or vendored UVM in the repo (see uvm_smoke Makefi...

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Basic Testbench (Directed Te"]

T2["Evolution to UVM+SV Testbenc"]

T1 --> T2

T3["Toolchain (Verilator, UVM_HO"]

T2 --> T3

Understand basic testbench patterns (directed tests, pin wiggling), the evolution to a UVM+SV testbench (agents, sequences, drivers, monito...

Overview

- Module 2 builds on Module 1 (spec → RTL) by focusing on ****how we verify****:

How to learn this module

Suggested learning path

- Complete Module 1 — run ``spec_to_rtl`` and understand spec → RTL → directed check.
- Read this module's Design Architecture and Verification sections before ``uvm_smoke``.
- Set UVM_HOME (or use vendored UVM under ``tools/``) — see Command Reference.
- Run ``uvm_smoke`` and trace one transaction from sequence → driver → DUT → monitor → scoreboard.

Design architecture

1. DUT and interface

- `simple_register.v`: ``enable``, ``d``, ``q`` — minimal storage for learning UVM data flow.
- `reg_if`: virtual interface bundling ``clk``, ``rst_n``, ``enable``, ``d``, ``q`` for the agent.

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
  subgraph uvm["UVM testbench"]
    SEQ["RegSequence"]
    DRV["RegDriver"]
    MON["RegMonitor"]
    SB["RegScoreboard"]
  end
```

2. UVM component hierarchy (1/2)

- RegTransaction → one write operation (data + enable + observed q).
- RegSequence → stream of directed transactions.
- RegDriver / RegMonitor → drive pins / sample pins each cycle.
- RegScoreboard → expected queue vs observed.

Verification Architecture

Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
T["RegTransaction\n(data, enable)"] --> SEQ["Sequence\n0x00..0xFF"]
SEQ --> DRV["Driver\nwiggles enable, d"]
DRV --> DUT["DUT q"]
DUT --> MON["Monitor\nsamples q"]
MON --> SB["Scoreboard\nexpected vs observed"]

2. UVM component hierarchy (2/2)

- RegAgent, RegEnv, RegTest — standard UVM layering used again in Modules 4, 6, 8.

3. Build and simulation flow

- Verilator compiles SV + UVM + C++; ``+UVM_TESTNAME`` selects the test.
- Phases: build → connect → run (objections hold simulation until sequence completes).

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SEQ["RegSequence"] --> DRV["RegDriver"]

DRV --> DUT["register"]

DUT --> MON["RegMonitor"]

MON --> SB["RegScoreboard"]

SB --> DONE["UVM test done"]

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph uvm["UVM testbench"]

SEQ["RegSequence"]

DRV["RegDriver"]

MON["RegMonitor"]

SB["RegScoreboard"]

Module 2: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

T["RegTransaction\n(data, enable)"] --> SEQ["Sequence\n0x00..0xFF"]

SEQ --> DRV["Driver\nwiggle enable, d"]

DRV --> DUT["DUT q"]

DUT --> MON["Monitor\nsample q"]

MON --> SB["Scoreboard\nexpected vs observed"]

Module 2: stimulus, observation, and checking.

RegTransaction — UVM item

```
/**
 * Module 2: UVM smoke test (Verilator)
 *
 * Purpose:
 * - Provide a *self-contained* UVM test that compiles and runs on
Verilator
 * - Demonstrate the core UVM structure used later for UART/SPI/I2C:
 *   transaction → sequence → driver/monitor → scoreboard
 */

`include "uvm_macros.svh"
import uvm_pkg::*;

`timescale 1ns/1ps

//
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator + UVM_HOME compile RTL, interface, and test_*.sv
- UVM phases: build → connect → run_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test_* (see module2 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

Example directory layout (1)

- DUT: dut/simple_register.v — tiny register (clk, rst_n, enable, d, q).
- Interface: reg_if — connects testbench to DUT.
- Transaction: RegTransaction — data + enable + observed_q.
- Sequence: RegSequence — produces 5 directed transactions (0x00, 0x01, 0x55, 0xAA, 0xFF).
- Driver: RegDriver — drives enable and d from transaction; waits for clock.

Example directory layout (2)

- Monitor: RegMonitor — samples enable, d, q each cycle; writes transaction to scoreboard.
- Scoreboard: RegScoreboard — compares expected vs observed; reports matches/mismatches.
- Test: RegTest — builds env, queues expected values, starts sequence, objections.

Makefile — UVM + Verilator

```
# Makefile for Module 2 UVM smoke test
#
# Mirrors the style used in:
# - tools/learn_uvm2017_sv_verilator/module3/tests/uvm_tests/Makefile
#
# Usage:
#   make SIM=verilator TEST=test_uvm_smoke
#

SIM ?= verilator
TEST ?= test_uvm_smoke

# Prefer an externally-installed UVM, but fall back to the repo's
# vendored UVM.
UVM_HOME ?= $(shell echo $$UVM_HOME)
# From module2/examples/uvm_smoke, repo root is ../../..
UVM_HOME_FALLBACK := ../../../../tools/uvm-2017/1800.2-2017-1.0
```

Key files to study

Open these in the repo

- ``module2/examples/uvm_smoke/dut/simple_register.v`` — tiny DUT
- ``module2/examples/uvm_smoke/test_uvm_smoke.sv`` — transaction, agent, scoreboard
- ``module2/examples/uvm_smoke/Makefile`` — Verilator + UVM build

Verification & testing methods

1. Transaction-level verification

- Stimulus is a transaction (what to write), not raw pin wiggles in `initial` blocks.
- The monitor reconstructs what happened on the bus; the scoreboard compares to expected.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SEQ["RegSequence"] --> DRV["RegDriver"]

DRV --> DUT["register"]

DUT --> MON["RegMonitor"]

MON --> SB["RegScoreboard"]

SB --> DONE["UVM test done"]

2. Directed regression in uvm_smoke

- Fixed sequence: 0x00, 0x01, 0x55, 0xAA, 0xFF — repeatable, easy to debug.
- Pass criteria: scoreboard reports matches, zero mismatches.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SEQ["RegSequence"] --> DRV["RegDriver"]

DRV --> DUT["register"]

DUT --> MON["RegMonitor"]

MON --> SB["RegScoreboard"]

SB --> DONE["UVM test done"]

3. Skills you reuse on UART/SPI/I²C

- Same agent pattern; only the transaction and protocol timing in driver/monitor change.
- Read UVM log lines: DRIVER, MONITOR, SCOREBOARD, final summary.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SEQ["RegSequence"] --> DRV["RegDriver"]

DRV --> DUT["register"]

DUT --> MON["RegMonitor"]

MON --> SB["RegScoreboard"]

SB --> DONE["UVM test done"]

Syllabus topics

1. Basic Testbench (Directed Tests, Pin Wiggling)

- Directed test: Drive specific inputs (e.g., reset, then a fixed sequence of data) and check expected...
- Pin wiggling: The testbench directly drives and samples DUT pins (clock, reset, data, enable) — no...
- Relation to Module 1: The spec_to_rtl example used a C++ harness to drive clk/rst_n/enable and check...

2. Evolution to UVM+SV Testbench

- Transaction (``uvm_sequence_item``): Represents “one operation” (e.g., one write to a register: data...
- Sequence: Produces a stream of transactions (e.g., a few directed values: 0x00, 0x01, 0x55, 0xAA, 0...
- Driver: Gets transactions from the sequencer and drives the DUT interface (e.g., set enable and d o...
- Monitor: Observes the DUT interface and turns pin activity into transactions, then sends them to th...
- Scoreboard: Compares “expected” (what we sent) vs “observed” (what the monitor saw). Reports matche...
- Agent: Groups driver, monitor, and sequencer for one interface. Env: Contains agent(s) and scoreboa...

3. Toolchain (Verilator, UVM_HOME, Make)

- Verilator: Compiles SystemVerilog (including UVM) + C++ into an executable. Key flags: ``-sv``, ``--ti...`
- UVM_HOME: Points to the UVM library root; the build uses ``$(UVM_HOME)/src/uvm_pkg.sv`` and ``+incdir+...`
- Make: One target to compile (Verilator + make in `obj_dir`), one to run (``./obj_dir/Vtest_uvm_smoke +...`

Command reference highlights

Environment checks

- verilator --version

Build and run uvm_smoke

- `cd module2/examples/uvm_smoke`

Module script (from repo root)

- `./scripts/module2.sh --check # Environment + UVM + example dirs`

Hands-on examples

Module 2 self-check

```
$ ./scripts/module2.sh --help
```

Terminal: module2_check

```
[[0;36m===== [[0m
[[0;36mModule 2: Demo commands (UVM smoke) [[0m
[[0;36m===== [[0m

From repo root:

# 1) Environment sanity:
verilator --version
make --version
echo "$UVM_HOME"

# 2) Run the uvm_smoke example (inside the example directory):
cd module2/examples/uvm_smoke
make SIM=verilator TEST=test_uvm_smoke

# 3) Inspect logs and waveforms:
ls obj_dir
# If a VCD is produced:
ls *.vcd
# Then open in your waveform viewer, e.g.:
gtkwave *.vcd # if available

[[1;33m[INFO] [[0m See module2/EXAMPLES.md and module2/examples/uvm_smoke/README.md for more.
```

Example 1: UVM smoke test

- UVM transaction, sequence, driver, monitor, and scoreboard on a tiny register DUT
- How objections start and end the test in ``run_phase``
- Interpreting DRIVER / MONITOR / SCOREBOARD messages and match counts

Demo: UVM smoke test

```
$ cd module2/examples/uvm_smoke && make SIM=verilator  
TEST=test_uvm_smoke
```

Terminal: ex01_uvm_smoke

From the repo root:

```
^^^ bash  
cd module2/examples/uvm_smoke
```

Key concepts

1. Transaction-level TB

- Stimulus is a transaction (what to do), not raw pin toggles in one big ``initial`` block.
- Driver converts transactions to pin activity;
monitor converts pins back to transactions.

2. Scoreboard closure

- Expected queue vs observed values — same pattern for UART/SPI/I²C UVM modules.

Common pitfalls

Skipping monitor or scoreboard

- Mistake: Only driving the DUT and eyeballing waves.
- Correct: Monitor + scoreboard automate expected vs observed.
- Why: Scales when you add dozens of tests.

UVM_HOME not set

- Mistake: Build fails with missing ``uvm_pkg.sv``.
- Correct: Export UVM_HOME or use vendored UVM under ``tools/``.
- Why: Compiler cannot find UVM includes.

Practice & assessment

What you should know

- Explain basic testbench (directed tests, pin wiggling) and how it evolves into UVM (transaction, se...
- Describe the role of transaction, sequence, driver, monitor, scoreboard, agent, env, and test in a...
- Run the uvm_smoke example (make SIM=verilator TEST=test_uvm_smoke) and interpret UVM output.
- Use the toolchain: Verilator, UVM_HOME, Make; know why ``-sv``, ``--timing``, and include paths are nee...
- Be ready for Module 3: UART protocol + RTL + basic (non-UVM) testbench.

Exercises

- Run uvm_smoke
- Traceability
- Change the sequence
- Optional: Waveforms

Assessment checklist

- Can explain basic TB vs UVM (transaction, sequence, driver, monitor, scoreboard).
- Can describe the role of UVM_HOME, Verilator flags, and Make in building/running a UVM test.
- Can run uvm_smoke and interpret the output.
- Ready to move to Module 3 (UART protocol + RTL + basic testbench).

Summary & next steps

- Understand basic testbench patterns (directed tests, pin wiggling), the evolution to a UVM+SV testb...
- Complete module2/CHECKLIST.md
- Review module2/EXAMPLES.md and run each lab
- Next: UART